🔹 1. Components

🔍 Summary:

Components are the **building blocks** of a React UI. They are JavaScript functions or classes that return JSX.

Two types:

- Functional Components (modern)
- Class Components (legacy)

💬 Interview Q&A:

Q1: What is a component in React? A1: A self-contained, reusable piece of UI that returns JSX and optionally manages state or props.

Q2: Example of functional component:

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

Q3: Can a component render another component? A3: Yes. Components can be composed to build complex UIs.

Q4: How do components help in reusability? A4: They encapsulate logic and structure, making the code modular and reusable across the app.

🔹 2. Props & State

🔍 Summary:

- **Props**: Passed from parent to child, read-only
- **State**: Local, mutable data managed by the component itself

---

💬 Interview Q&A:

Q1: What are props in React? A1: Inputs passed to components from parent components.

Q2: What is state in React? A2: Internal data that can change over time and trigger re-renders.

Q3: Example:

```
function Counter({ title }) {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h3>{title}</h3>
      <button onClick={() => setCount(count + 1)}>Click {count}</button>
    </div>
  );
}
```

Q4: Can props be changed inside a child component? A4: No, props are read-only. You must pass a callback to update them from the parent.

🔹 3. Hooks

🔍 Summary:

Hooks let you use state, lifecycle, and context in **functional components**.

Common hooks:

- `useState`
- `useEffect`
- `useContext`
- `useMemo`
- `useCallback`
- `useRef`

💬 Interview Q&A:

Q1: What is the purpose of hooks in React? A1: Hooks allow functional components to have state and lifecycle features without needing classes.

Q2: Example of `useState` and `useEffect`:

```
const [name, setName] = useState('React');

useEffect(() => {
  console.log(`Hello, ${name}`);
}, [name]);
```

Q3: Can hooks be called inside loops or conditions? A3: ❌ No. Hooks must be called at the top level of a component to preserve consistent call order.

Q4: What's a custom hook? A4: A reusable function that uses other hooks internally.

🔹 4. Lifecycle Methods (Class Components)

🔍 Summary:

In class components, lifecycle methods are functions that run during specific stages of a component's life.

Key methods:

- `componentDidMount`
- `componentDidUpdate`
- `componentWillUnmount`
- `shouldComponentUpdate`

💬 Interview Q&A:

Q1: What is a lifecycle method? A1: A special method in class components that gets called at different phases (mounting, updating, unmounting).

Q2: Example of `componentDidMount`:

```
class App extends React.Component {
  componentDidMount() {
    console.log('Mounted!');
  }
  render() {
    return <div>Hello</div>;
```

```
  }
}
```

Q3: What's the functional equivalent of `componentDidMount`? A3:

```
useEffect(() => {
  console.log('Mounted');
}, []);
```

Q4: When would you use `componentWillUnmount`? A4: To clean up (e.g., cancel timers, unsubscribe from services).

🔹 5. Context API

🔍 Summary:

The **Context API** lets you share global data (like themes, auth status, language) across your React component tree **without passing props manually at every level**.

💬 Interview Q&A:

Q1: What is the Context API in React? A1: It's a way to create and provide global data that can be consumed by any component, avoiding prop drilling.

Q2: Steps to use context?

1. Create context → `const ThemeContext = React.createContext();`
2. Provide context → Wrap root with `<ThemeContext.Provider value={...}>`
3. Consume context → `useContext(ThemeContext)`

Q3: Example:

```
const ThemeContext = React.createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
```

```
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>Using {theme} theme</div>;
}
```

Q4: When should Context not be used? A4: For high-frequency updates (like keystrokes), as it can trigger unnecessary re-renders.

🔹 6. State Management

🔍 Summary:

React provides local state via `useState`, but **global state** is managed using:

- Context API (simple)
- Redux / Redux Toolkit (complex)
- Zustand, Recoil, Jotai, MobX (modern alternates)

💬 Interview Q&A:

Q1: Why do we need state management libraries? A1: When multiple components need access to or control over shared state (e.g., user session, cart, theme).

Q2: When should you use Context API vs Redux? A2:

- Use **Context API** for simple global values
- Use **Redux Toolkit** for complex logic, async actions, or deeply nested state

Q3: How does Redux work at a high level? A3:

- Actions → describe change
- Reducers → apply change
- Store → central state object
- Components → dispatch and subscribe

🔹 7. Component Styling

🔍 Summary:

React supports many styling approaches:

- Inline styles
- CSS Modules
- Styled Components (CSS-in-JS)
- Tailwind, Emotion, Sass, etc.

💬 Interview Q&A:

Q1: What are different ways to style components in React? A1:

- Inline styles: `style={{ color: 'red' }}`
- CSS Modules: `import styles from './file.module.css'`
- CSS-in-JS: styled-components

Q2: Example using CSS Modules:

```
import styles from './Button.module.css';

function Button() {
  return <button className={styles.primary}>Click Me</button>;
}
```

Q3: Why prefer CSS Modules or Tailwind over global CSS? A3: Scoped styles prevent conflicts and make component reuse easier.

Q4: Example using Styled Components:

```
import styled from 'styled-components';
const Button = styled.button`
  color: white;
  background: blue;
`;

<Button>Click</Button>
```

🔹 8. Form Handling

🔍 Summary:

React treats forms as controlled components — values are bound to state.

Advanced form libraries:

- `react-hook-form` (lightweight, performant)
- `Formik` (robust but heavier)

💬 Interview Q&A:

Q1: What is controlled vs uncontrolled input? A1:

- **Controlled**: value managed by React via `useState`
- **Uncontrolled**: DOM manages value using `ref`

Q2: Controlled form example:

```
function Form() {
  const [email, setEmail] = useState('');
  return (
    <input
      type="email"
      value={email}
      onChange={e => setEmail(e.target.value)}
    />
  );
}
```

Q3: How does `react-hook-form` work?

```
import { useForm } from 'react-hook-form';

function App() {
  const { register, handleSubmit } = useForm();
  const onSubmit = data => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} />
      <button type="submit">Submit</button>
```

```
    </form>
  );
}
```

Q4: Why prefer `react-hook-form` over manually managing state? A4: It's more performant (less re-renders), and provides built-in validation, error handling, and cleaner code.

🔹 9. Error Boundaries

🔍 Summary:

Error Boundaries are React components that **catch runtime errors** in their child component tree during rendering, lifecycle methods, and constructors — and render a fallback UI instead of crashing the app.

Only available in **class components** (as of React 18).

💬 Interview Q&A:

Q1: What are Error Boundaries in React? A1: Components that catch JavaScript errors in children and display a fallback UI.

Q2: How to create one?

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error("Logged error:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>;
    }
    return this.props.children;
```

```
  }
}
```

Usage:

```
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

Q3: Can hooks be used to create error boundaries? A3: Not directly. Hooks can't catch errors like rendering issues. Use `ErrorBoundary` class instead.

🔹 10. Redux Toolkit

🔍 Summary:

Redux Toolkit is the official, recommended way to use Redux. It simplifies store setup, reduces boilerplate, and integrates well with TypeScript and async actions via `createAsyncThunk`.

💬 Interview Q&A:

Q1: Why Redux Toolkit over plain Redux? A1: It reduces boilerplate and provides:

- `configureStore` with sensible defaults
- `createSlice` for reducers + actions
- `createAsyncThunk` for async logic

Q2: Basic counter slice example:

```
import { createSlice, configureStore } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: {
    increment: state => state + 1,
    decrement: state => state - 1,
  }
});
```

```
const store = configureStore({ reducer: { counter: counterSlice.reducer } });
```

Q3: What is `createAsyncThunk` used for?

```
export const fetchUser = createAsyncThunk('user/fetch', async (id) => {
  const res = await fetch(`/api/user/${id}`);
  return await res.json();
});
```

Q4: Does Redux Toolkit replace Context API? A4: No. Context API is good for light state; Redux Toolkit is better for complex/global/shared state across multiple components.

🔹 11. Server Components (React 18+)

🔍 Summary:

Server Components are a new experimental feature allowing components to **run only on the server** and send rendered content to the client. They're stateless, fast, and reduce JS bundle size.

Used mainly in frameworks like **Next.js (App Router)**.

💬 Interview Q&A:

Q1: What are Server Components? A1: React components that run on the server and are never sent to the browser. They return serialized UI and improve performance.

Q2: Key differences from client components:

- No state, effects, or event listeners
- Can access DB/filesystem directly
- Smaller client bundle

Q3: Syntax example (Next.js):

```
// app/page.tsx (runs on server)
export default async function Page() {
  const data = await getDataFromDB();
```

```
  return <div>{data.title}</div>;
}
```

Q4: When to use Server Components? A4: For data-heavy, read-only components where interactivity isn't needed on the client.

🔹 12. Styling Libraries

🔍 Summary:

React supports many styling libraries:

- **Styled Components** (CSS-in-JS)
- **Emotion**
- **Tailwind CSS** (utility-first)
- **Sass**
- **Chakra UI / Material UI** (component libraries)

💬 Interview Q&A:

Q1: What's the benefit of using styled-components or Emotion? A1: Styles are co-located with components, scoped automatically, and dynamic (accept props).

Q2: Example using styled-components:

```
import styled from 'styled-components';

const Button = styled.button`
  background: ${props => props.primary ? 'blue' : 'gray'};
`;

<Button primary>Click</Button>
```

Q3: What is Tailwind CSS and why is it popular? A3: A utility-first CSS framework with predefined classes for styling (e.g., `p-4`, `bg-blue-500`). Offers rapid development and consistent design.

Q4: How does Emotion differ from Styled Components? A4: Emotion is more flexible and supports both string + object styles, while styled-components focuses on tagged template literals.

🔹 13. Custom Hooks

🔍 Summary:

Custom Hooks allow you to **reuse stateful logic** across multiple components. They're just normal functions that use built-in hooks like `useState`, `useEffect`, etc.

Convention: Hook names must start with `use`.

💬 Interview Q&A:

Q1: What is a custom hook in React? A1: A function that uses other hooks internally to encapsulate and reuse logic between components.

Q2: Simple custom hook example (title):

```
function useDocumentTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}

// Usage:
useDocumentTitle("My Page");
```

Q3: Why are custom hooks useful? A3: To keep components clean and DRY by abstracting repetitive logic like form handling, fetching data, etc.

Q4: Can a custom hook use another custom hook? A4: ✅ Yes. Hooks are composable.

🔹 14. Forwarding Refs

🔍 Summary:

Refs are used to access DOM elements directly in React. **Forwarding refs** lets you pass a `ref` through a component to one of its child DOM nodes.

Use case: Wrapping native input fields, exposing `.focus()` or scroll behavior.

💬 Interview Q&A:

Q1: What is `forwardRef` in React? A1: It allows a parent to access a child DOM node through a wrapper component using `ref`.

Q2: Example:

```
const Input = React.forwardRef((props, ref) => (
  <input ref={ref} {...props} />
));

function App() {
  const inputRef = useRef();
  return (
    <>
      <Input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

Q3: When would you use forwardRef? A3: When you're building reusable UI libraries or components that wrap other elements and still want to expose their refs.

🔹 15. Lazy Loading

🔍 Summary:

Lazy loading in React means **deferring the loading of components** until they are needed (e.g., on scroll, route change). Helps reduce initial JS bundle size.

React provides `React.lazy` + `Suspense` for component-level code splitting.

💬 Interview Q&A:

Q1: What is React lazy loading? A1: It's a way to dynamically import components only when needed, improving load performance.

Q2: Example with `React.lazy`:

```jsx
const About = React.lazy(() => import('./About'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <About />
    </Suspense>
  );
}
```

Q3: Can lazy loading be used with routes (e.g., React Router)? A3: ✅ Yes. It's a common practice:

```jsx
<Route path="/about" element={
  <Suspense fallback={<Loading />}>
    <About />
  </Suspense>
} />
```

Q4: Does lazy loading work on non-components like utilities? A4: No. `React.lazy` only works with default exports that return React components.

🔹 16. useEffect

🔍 Summary:

`useEffect` is a hook to handle **side effects** in function components:

- API calls
- Subscriptions
- DOM manipulation
- Event listeners

You can think of it as a combination of `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.

💬 Interview Q&A:

Q1: What is `useEffect` used for? A1: To perform side effects like fetching data, setting up subscriptions, timers, etc.

Q2: Basic usage:

```
useEffect(() => {
  console.log('Runs on mount');
}, []);
```

Q3: How to cleanup in `useEffect`?

```
useEffect(() => {
  const timer = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(timer); // cleanup
}, []);
```

Q4: What happens if no dependency array is passed? A4: The effect runs after **every render**.

Q5: Can you have multiple `useEffect` calls? A5: ✅ Yes. You can organize logic by grouping side effects independently.

🔹 17. useMemo / useCallback

🔍 Summary:

These hooks help **optimize performance** by **memoizing values or functions**, preventing unnecessary re-computation or re-renders.

- `useMemo`: Memoizes a **value**
- `useCallback`: Memoizes a **function**

💬 Interview Q&A:

Q1: What is `useMemo`? When should you use it? A1: `useMemo` caches a **computed value** between renders unless dependencies change.

```
const expensiveValue = useMemo(() => computeHeavy(num), [num]);
```

Q2: What is `useCallback`? A2: Returns a memoized function to avoid creating a new one every render.

```
const handleClick = useCallback(() => doSomething(id), [id]);
```

Q3: Difference between `useMemo` and `useCallback`? A3:

- `useMemo`: returns a memoized **value**
- `useCallback`: returns a memoized **function**

Q4: When are they useful? A4: When passing functions to child components or working with expensive computations in large trees.

🔹 18. State Lifting

🔍 Summary:

State Lifting is the process of moving shared state **up** to the closest common ancestor so that multiple sibling components can access and modify it via props.

💬 Interview Q&A:

Q1: What is state lifting in React? A1: Moving state up to a parent component so it can be shared and controlled by multiple children.

Q2: Example:

```
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>
      <ChildA count={count} />
      <ChildB updateCount={() => setCount(c => c + 1)} />
    </>
  );
}
```

Q3: Why is lifting state important? A3: It promotes single source of truth and enables consistent, shared state logic.

Q4: Alternatives to lifting state? A4: Context API, Redux, Zustand, or any global state manager.

🔹 19. JSX

🔍 Summary:

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code in JavaScript, which is compiled to `React.createElement()`.

It improves **readability** and **developer experience**.

💬 Interview Q&A:

Q1: What is JSX? A1: A syntax extension for JavaScript used with React to describe UI elements. It's transpiled into plain JS.

Q2: Can you write logic inside JSX? A2: ✅ Yes, using expressions inside `{}`:

```
<div>{count > 0 ? "Yes" : "No"}</div>
```

Q3: How does JSX work under the hood? A3: It's compiled into:

```
React.createElement('div', null, 'Hello')
```

Q4: Can JSX have multiple elements at the top level? A4: ❌ Not directly — you must wrap them in a parent tag or `<> </>` (Fragment).

🔹 20. Hydration

🔍 Summary:

Hydration is the process where React takes over **server-rendered HTML** and attaches event handlers to make it interactive on the client side.

Common in frameworks like **Next.js** (SSR).

💬 Interview Q&A:

Q1: What is hydration in React? A1: It's how React attaches JS behavior to server-rendered HTML on the client.

Q2: When does hydration happen? A2: After SSR — browser receives static HTML and React takes over during client initialization.

Q3: Why is hydration important? A3: Combines benefits of SSR (SEO, fast load) with interactivity of React.

Q4: Example in Next.js:

```
export default function Page({ data }) {
  return <div>{data.title}</div>;
}

// getServerSideProps → HTML generated on server
// Hydration attaches events when React loads on browser
```
